

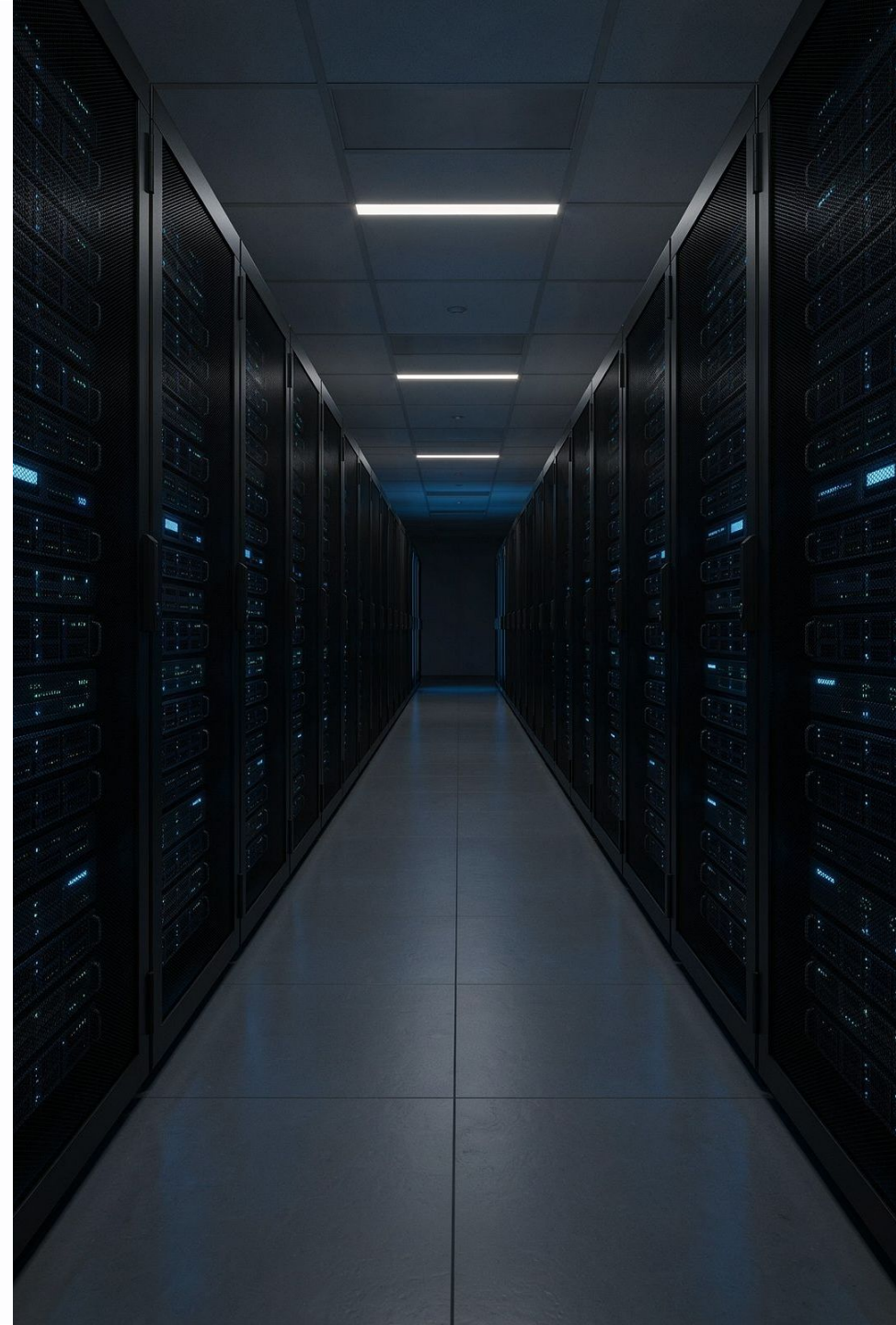
Day 1: Cloud Foundations, Architecture & Secure CI/CD

A practical, story-driven guide through the building blocks of modern cloud computing – from infrastructure basics to secure deployment pipelines and Infrastructure as Code. Built for developers and IT professionals who want to go beyond theory and understand how it all connects in the real world.

5 MODULES

HANDS-ON LEARNING

REAL-WORLD EXAMPLES



Today's Roadmap

What We'll Cover Today

Five focused modules that build on each other — start with cloud basics, understand architecture, then layer in CI/CD, security, and finally Infrastructure as Code. By the end, you'll have a clear mental model of how modern cloud systems are built, deployed, and secured.

01

Modern Cloud Computing

The "why" behind cloud — service models, deployment types, and shared responsibility

02

Cloud Architecture Fundamentals

Regions, networks, compute, storage — the physical and logical building blocks

03

CI/CD Fundamentals

Automating code from commit to production with pipelines and workflows

04

Secure CI/CD

Baking security into the pipeline — secrets, scanning, and security gates

05

Infrastructure as Code (Terraform)

Provisioning cloud resources with code — reproducible, versioned, automated



MODULE 1

Modern Cloud Computing

Cloud computing changed everything about how software is built and delivered. Instead of buying servers and waiting weeks for hardware, teams now spin up infrastructure in minutes. This module explains the core concepts that make that possible — and why it matters for every developer and IT professional today.

Traditional Infrastructure vs. The Cloud

Imagine opening a restaurant. In the old model, you'd buy the building, all the kitchen equipment, and staff — even on slow days. The cloud is like renting a commercial kitchen by the hour. You pay only for what you use, scale up on Friday nights, and let someone else maintain the ovens.



Traditional Infrastructure

- Buy and rack physical servers upfront
- Weeks to months to provision
- Pay for full capacity, even at idle
- You manage all hardware and software
- Fixed capacity — hard to scale fast



Cloud Infrastructure

- Provision virtual resources in minutes
- Pay only for what you consume
- Scale up or down on demand
- Provider manages physical hardware
- Global reach with a few clicks

The Shared Responsibility Model

When you move to the cloud, security becomes a partnership. Think of it like renting an apartment — the landlord (cloud provider) secures the building, locks, and plumbing. You're responsible for locking your own door and not leaving your laptop on the sidewalk.

Cloud Provider's Responsibility

- Physical data center security
- Hardware and network infrastructure
- Hypervisor and virtualization layer
- Global network backbone

Your Responsibility

- Identity and access management (IAM)
- Data encryption and classification
- Application-level security
- Operating system patching (IaaS)
- Network configuration and firewall rules

📌 The boundary shifts depending on the service model — the more managed the service, the more the provider handles.

Cloud Service Models: IaaS, PaaS, SaaS

Three levels of abstraction, three different relationships with the cloud. The higher you go, the less you manage — but also the less control you have. The right choice depends on your team's skill set and what you're trying to build.



IaaS — Infrastructure as a Service

You rent raw compute, storage, and networking. Like leasing a bare server in a remote data center. You install the OS, runtime, and everything else. **Example:** AWS EC2, Azure VMs, Google Compute Engine.



PaaS — Platform as a Service

You deploy your app; the platform handles the runtime, OS, and scaling. Like handing your code to a managed hosting service. **Example:** Heroku, AWS Elastic Beanstalk, Google App Engine.



SaaS — Software as a Service

You just use the software. No servers, no deployments. The provider handles everything. **Example:** Gmail, Salesforce, Microsoft 365, Slack.

Cloud Deployment Models

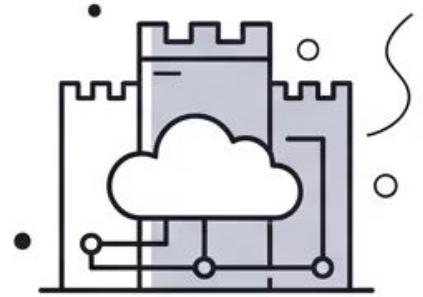
Not all workloads belong in the same place. A startup may go all-in on public cloud, while a hospital may keep patient records private and only push analytics to the cloud. Most enterprises today use a mix – and understanding the trade-offs is key.



PUBLIC CLOUD

**Shared infrastructure.
Fully remote. Managed by
provider. Low cost.**

PRIVATE CLOUD



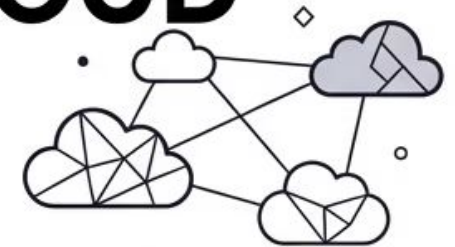
**Dedicated infrastructure.
On-premises or hosted.
High control.**

HYBRID CLOUD



**Mix of public & private.
Connected via VPN/direct
link. Workload flexibility.**

MULTI-CLOUD



**Use 2+ public providers.
Avoid lock-in. Increase
resilience.**



Module 1 Wrap-Up

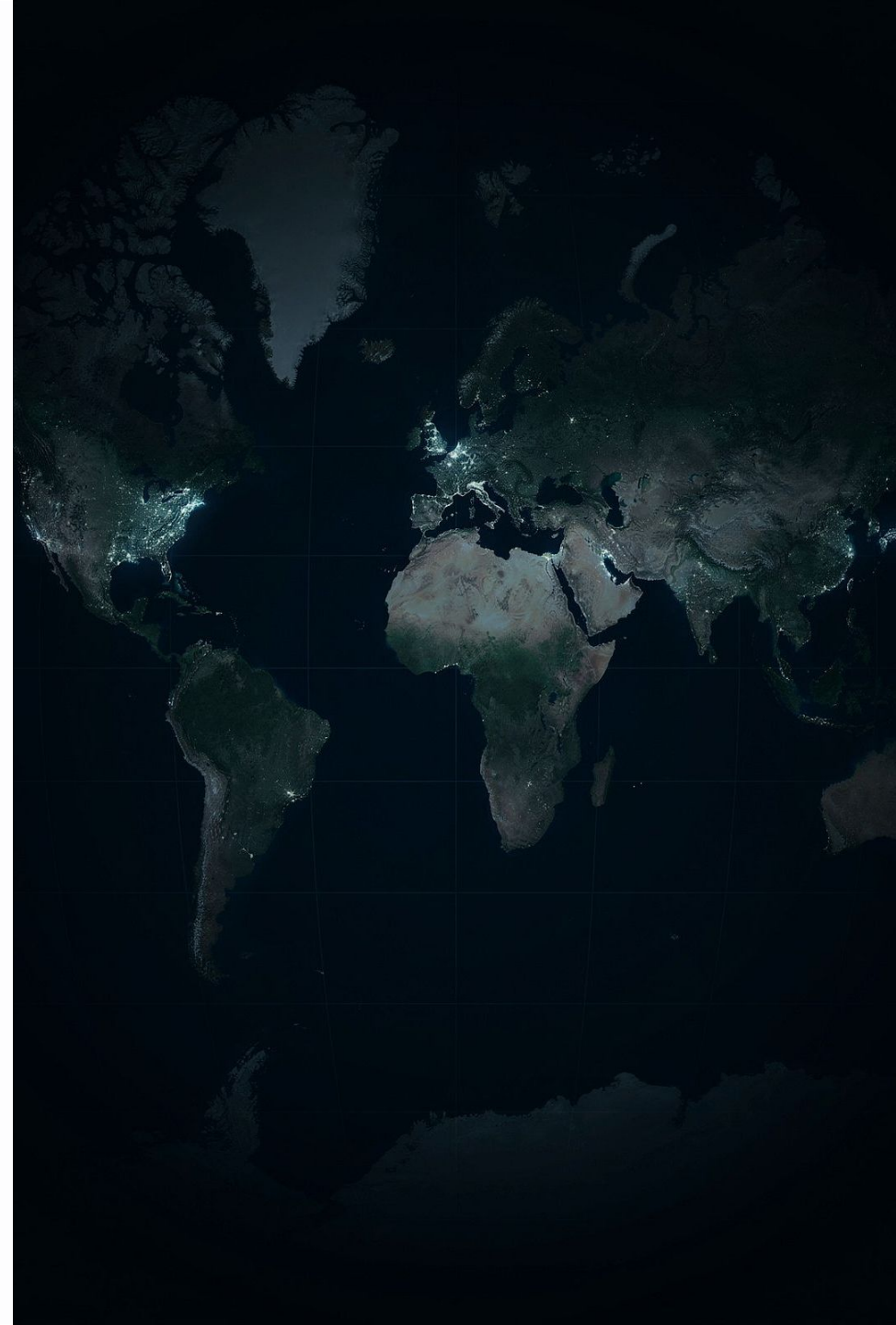
Cloud-Native Applications

Cloud-native apps are designed from the ground up to take advantage of the cloud — built as microservices, packaged in containers, orchestrated with Kubernetes, and deployed via automated pipelines. They're elastic, resilient, and observable. This is the target architecture for everything else we build today.

MODULE 2

Cloud Architecture Fundamentals

Understanding cloud architecture means knowing how the physical and logical layers fit together. Before you deploy a single VM or container, you need to understand where your workloads live, how they communicate, and who can access them. This module builds the mental map every cloud engineer needs.



Regions, Availability Zones & the Global Network

Think of cloud infrastructure like a chain of hospitals. A **Region** is a city with multiple hospital campuses. Each campus is an **Availability Zone (AZ)** — isolated from the others so a fire in one doesn't affect the rest. Designing across AZs means your app stays up even when one zone fails.

1

Region

A geographic area containing 2+ AZs.
Examples: *us-east-1* (N. Virginia), *eu-west-1* (Ireland). Choose based on latency and data residency laws.

2

Availability Zone

One or more physically isolated data centers within a region. Each has independent power, cooling, and networking. Spread workloads across AZs for high availability.

3

Edge Locations

CDN endpoints that cache content close to end users. Reduces latency for global audiences. Used by AWS CloudFront, Azure CDN, and Google Cloud CDN.

Virtual Networks, Subnets & Security Groups

A **Virtual Private Cloud (VPC)** is your own private slice of the cloud network — like a gated office building. Inside, you create **subnets** to separate floors (public web tier, private app tier, isolated database tier). **Security Groups** act as smart firewalls on each door, allowing only the right traffic in and out.

Network Segmentation Pattern

- **Public Subnet** — Load balancers, bastion hosts (internet-facing)
- **Private Subnet** — App servers, API services (no direct internet access)
- **Isolated Subnet** — Databases, secrets (no outbound internet)

Security Group Rules (Example)

- Allow port 443 (HTTPS) from 0.0.0.0/0 → web tier
- Allow port 8080 from web tier SG → app tier
- Allow port 5432 from app tier SG → database only
- Deny all other inbound by default

Load Balancers, IAM, Compute & Storage

These four services form the operational backbone of almost every cloud workload. Get comfortable with them — you'll use them in every module that follows.



Load Balancers

Distribute incoming traffic across multiple instances. Prevent any single server from being overwhelmed. Provide health checks — dead instances are automatically removed.



IAM

Identity and Access Management. Controls *who* (users, services, roles) can do *what* (read, write, delete) on *which* resource. Principle: least privilege always.



Compute

Virtual Machines (EC2, Azure VMs), containers (ECS, AKS), and serverless (Lambda, Functions). Match compute type to workload size and pattern.



Storage

Object storage (S3, Blob), block storage (EBS, Managed Disks), file storage (EFS, Azure Files). Each optimized for different access patterns and costs.



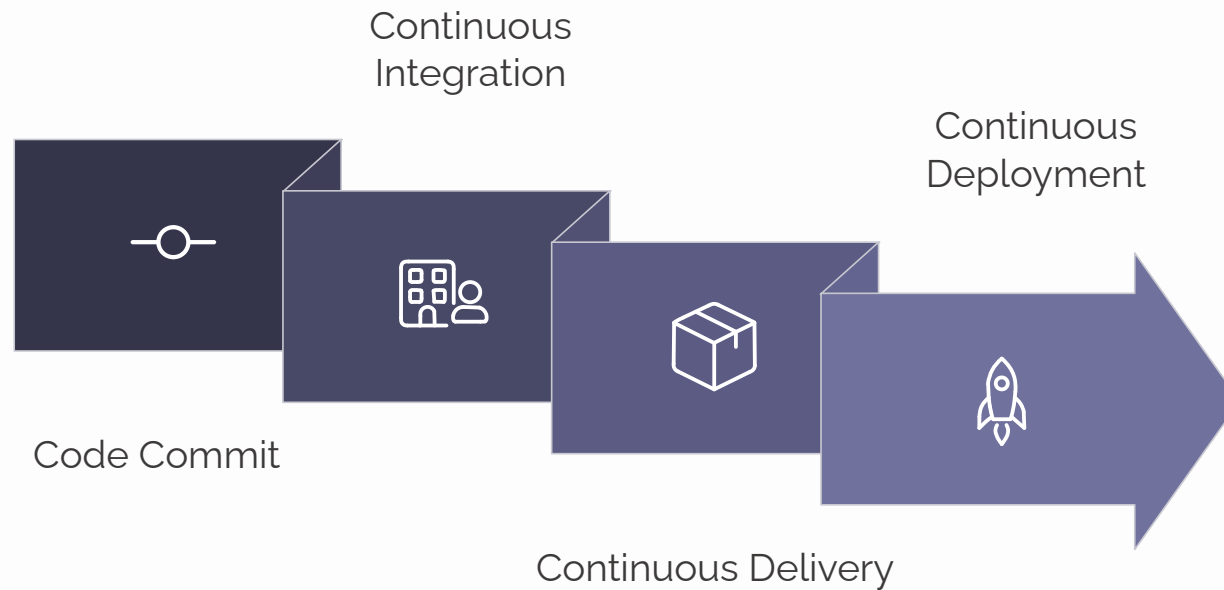
MODULE 3

CI/CD Fundamentals

Before CI/CD, releasing software was like moving apartments once a year — stressful, risky, and full of surprises. CI/CD turns that into a smooth, daily routine where small changes ship safely and automatically. This module explains how the pipeline works and why every modern team runs one.

From Code Commit to Production: The CI/CD Pipeline

The CI/CD pipeline is the assembly line of software delivery. Every time a developer pushes code, the pipeline kicks off automatically — building, testing, and deploying without human toil. Here's how the stages connect:



The key difference: **Continuous Delivery** requires a manual approval button before production. **Continuous Deployment** removes that button entirely — every passing build ships automatically.

Git Workflow, Artifacts & the Build Pipeline

A well-designed Git workflow is the foundation of a healthy pipeline. Feature branches prevent chaos; pull requests enforce code review; main/trunk stays always deployable. Once code is merged, the build pipeline turns raw source into a versioned, testable artifact.

Git Workflow (Trunk-Based)

- Developers push short-lived feature branches
- Pull Request triggers the CI pipeline
- Code review + passing tests = merge to main
- Main branch is always production-ready

Build Pipeline Output

- **Artifact:** compiled binary, Docker image, or ZIP
- Stored in an **Artifact Repository** (e.g., JFrog Artifactory, AWS ECR, GitHub Packages)
- Tagged with Git commit SHA for full traceability
- Same artifact promoted through all environments — no rebuilds



MODULE 4

Secure CI/CD

Speed without security is a liability. This module shows how to weave security directly into your pipeline so vulnerabilities are caught before they reach production — not after a breach. Think of it as adding a metal detector to your assembly line.

Securing the Pipeline: From Secrets to Security Gates

A secure CI/CD pipeline has defense in depth — multiple layers of checks, each catching different types of risk. Here's how the key practices fit together into a comprehensive security strategy:

Secrets Management

Never hardcode API keys or passwords in code. Use a secrets manager (HashiCorp Vault, AWS Secrets Manager). Inject secrets at runtime as environment variables — rotated automatically, audited centrally.

Static Code Analysis (SAST)

Scan source code for security vulnerabilities before it's even compiled. Tools like SonarQube or Semgrep catch SQL injection, hardcoded credentials, and insecure patterns on every PR.

Dependency & Container Scanning

Open-source libraries and base container images often carry known CVEs. Tools like Snyk, Trivy, or Gype scan dependencies and Docker images automatically. Fail the build if critical vulnerabilities are found.

Security Gates

Define pass/fail thresholds at each pipeline stage. Example: block deployment if any Critical CVE is found, or if code coverage drops below 80%. Security gates make policy machine-enforced, not human-dependent.



MODULE 5

Infrastructure as Code with Terraform

What if you could describe your entire cloud infrastructure in a text file and recreate it perfectly in any environment, any time? That's Infrastructure as Code. Terraform is the industry-standard tool for doing exactly that — and it works across AWS, Azure, GCP, and beyond.

Terraform's Core Building Blocks

Terraform uses a declarative language (HCL) where you describe the *desired state* of your infrastructure, and Terraform figures out how to get there. Think of it like ordering from a menu — you say what you want, not how to cook it.

Providers

Plugins that talk to cloud APIs. `aws`, `azurerm`, `google`. Define them once at the top of your config.

Resources

The actual cloud objects to create — a VM, a VPC, an S3 bucket. The core of every Terraform config.

Variables

Parameterize your configs. Pass in environment-specific values (dev vs. prod) without changing the code itself.

Outputs

Expose values from your infrastructure (like an IP address or bucket name) to be used by other modules or pipelines.

State File

Terraform's memory. Tracks what it has created. Store remotely (S3 + DynamoDB lock) — never commit to Git.

Modules

Reusable packages of Terraform code. Build a VPC module once, use it in every project. Promotes consistency and DRY principles.

The Terraform Workflow & Day 1 Recap

The Terraform Workflow

1 terraform init

Download providers and set up the working directory

2 terraform plan

Preview what will be created, changed, or destroyed — no changes yet

3 terraform apply

Execute the plan and provision real infrastructure in the cloud

4 terraform destroy

Tear down everything — great for temporary environments and cost control

Day 1 Key Takeaways

- Cloud shifts capital expense to operational expense — pay for what you use
- Security is a shared model — know your boundary
- Architecture decisions (regions, AZs, subnets) affect availability and compliance
- CI/CD pipelines automate quality and speed of delivery
- Security must be built into the pipeline, not bolted on after
- Terraform lets you treat infrastructure like software — versioned, reviewed, automated

📌 The best time to add security and good architecture habits is Day 1 — not after your first incident.